CO1035: Game Concepts
Assignment 1 – Spheres.io

Assignment Information:

- **Course:** BSc Computing
- **Module Code:** CO1035
- **Module Title:** Game Concepts
- **Assignment Title:** Spheres.io
- **Type of Assignment:** 3D Game
- **Weight:** 40%
- **Assignment Release Day: 14/10/2026**
- **Assignment Deadline: 13/11/2026**
- **Feedback Deadline: 11/12/2026**

Submission Details:

- Submission is electronic **via Blackboard** through the available submission link.
- You should submit **a single zipped folder**, named Assignment1_[Username].zip (e.g. **Assignment1_AHadjiliasi1.zip**), that includes the following items:
  - The **.cpp** file of your project, named Assignment1_Username.cpp (e.g., **Assignment1_AHadjiliasi1.cpp**). For this assignment, **multiple file submission is NOT allowed**. Hence, your entire code should be included in one .cpp file and must compile without any compiler errors.
  - A **media folder** that includes only the extra media you used that are not part of the TL-Engine folder.
  - A **report** created using the provided report template.
  - The coursework **cover sheet document completed**.
- **Note:** Please do not include your whole project folder. If you submit the whole project folder, or your game fails to compile when tested by the marker, **your mark will be zero**.
- **If you have any questions, please discuss them with your Module Leader.**

Important Considerations

- **For this assignment, you are not permitted to use any category of AI tools.** AI tools can do a lot of things to make our lives easier. However, the goal of this assignment is to help you develop the necessary skills to think, analyze, and solve problems like a programmer. Hence, the use of AI tools in this assignment is strictly not permitted, and any case of AI use **will be treated as plagiarism**.
- **You are only allowed to use techniques and methods that were covered in class.** The use of methods, techniques, or anything that has not been covered in class is not permitted, unless there is a good reason. If a technique was deemed mandatory to be used, and it's coming from the internet, make sure to properly justify it and cite the source.
- **This is an individual work.** This means you are not allowed to collaborate with other students, nor to use any external help. After submission, the submissions are going through a similarity check. Note that tricks such as changing the variable names, leaving more spaces between the code, or even shuffling code can still be detected. **If you need help, please contact the module leader**.

Additional Support Available:

- Academic support for this assignment will be provided by contacting the module leader via email (AHadjiliasi1@uclan.ac.uk)
- Our library resources can be found in the library at the Student Hub, or via your subject librarian (via CyprusLibrary#uclan.ac.uk)
- If you have a disability, specific learning difficulty, long-term health or mental health condition, and have not yet advised us, or would like to review your support, **Student Support Officers (cyprusstudentsupport@uclancyprus.ac.cy)** can assist with reasonable adjustments and support. To find out more, you can visit the **Student Support Service** page of the Student Hub. You can also call +357 24694026, +357 24694108, or +357 24694073.
- For mental health and wellbeing support, please complete our online referral form or email the Psychological Wellbeing and Counselling Centre at UCLan Cyprus at wellbeing@uclancyprus.ac.cy.
- For any other support queries, please contact **Student Support** via CyprusStudentSupport@uclan.ac.uk.
- For consideration of Academic Integrity, please refer to detailed guidelines in our policy document. All assessed work should be genuinely your own work, and all resources fully cited.

Instructions

You must follow the specification to implement a simple 3D game inspired by **Katamari Damacy** and the popular "io" games. You are expected to complete this in your own time outside of scheduled labs. But you should ask the module leader or your lab tutor for advice if you are stuck. You **MUST USE TL-ENGINE** to implement this game. The specification for implementing this game can be found **at the end of this document**, and a template for producing your report can be found on Blackboard. However, **PLEASE READ THIS BRIEF IN FULL** for full guidance on completing and submitting this assessment. Do not diverge from the specification and instructions provided in this brief. If you do, then you will lose marks. Ask for clarification if you are unsure!

LEARNING OUTCOMES ASSESSED

- Describe key game concepts, e.g., genres, terminology.
- Use a game engine to create simple computer game prototypes.
- Apply mathematical techniques for analysis and reasoning about problems

CODING STYLE

- Your code MUST be properly indented and laid out so that it is readable.
- Your code should be commented appropriately; over-long lines of code or comments should be split up
- Variable names must be meaningful.
- You should have no "magic numbers" but instead make proper use of variables and constants.
- You should pay attention to the correct and appropriate use of data types.

REQUIREMENTS FOR HIGH MARKS

- You should make use of enumerated types to control the states within the game where appropriate.
- Utilize functions for collision detection and vector calculations; implement functions outside the main program as appropriate.
- Utilize arrays and loops where appropriate.
- Pay attention to the playability of the game as well as achieving the technical requirements.

PLAGIARISM

- This is an individual project and no group work is permitted.
- You will be held responsible if someone copies your work - unless you can demonstrate that you have taken reasonable precautions against copying.
- Remember the machines in the Games Lab have shared drives. You must either password-protect your work or else remove it from the hard drive each time you finish working on it. You may also consider using a change management system like GitHub.

SPHERES.IO Game Specifications

You must follow this specification to implement a simple 3D game inspired by the popular "io" games. Ask the module leader or your lab tutor for advice if you are stuck or unsure. You should implement the features described below in order. To be eligible for a mark within any classification, you must have completed all the features for all the previous classifications.

MEDIA RESOURCES

This assessment has an associated set of media files which can be found inside "**Assessment1Resources.zip**" on Blackboard. The model meshes included in the folder are:

Name:	Description:	Important Notes:
spheremesh.x	A sphere	Radius = 10 units
water.x	Water	-
island.x	The ground	200 x 10 x 200 units
sky.x	A skybox	-
minicube.x	A cube	5 x 5 x5 units

Please remember that all of the models and textures I supply are under license and must not be passed on to anyone else or used for any purpose other than your University work.

BASIC LAYOUT

a. Create a new TL-Engine project and, using the resources provided for this assignment, set up your scene as specified below before moving on to meeting the specified milestones.
b. Create a model using water.x and position it at (0, -5, 0).
c. Create an island using island.x and place it at (0, -5, 0). It should appear to be an island; its bottom half is submerged into the water, and the ground of its top half at 0 along the y-axis. This will enable you to position objects on the island more naturally. This will be the stage for your game.
d. Create a sphere model using spheremesh.x and position it at 0 along the x and z-axis, and high enough along the y-axis so that it appears to sit on the floor (hint: the radius of the sphere is 10 units, and its origin is at its center). This will serve as the player-controlled game object.
e. Create four cube models (cube1 at x = -80 and z = 80; cube2 at x = 80 and z = 80; cube3 at x = 80 and z = -80; and cube4 at x = -80 and z = -80) using minicube.x, and make sure they are positioned appropriately along the y-axis so that they appear to sit on the floor. All of the boxes must be axis-aligned (i.e., their edges are parallel with the axis).
f. Create a skybox using sky.x and position it at (0, -960, 0).
g. Add a top-down camera by creating a kManual camera at (0, 200, 0) and rotate it by 90 degrees about its local x-axis.

MILESTONE 1: THIRD CLASS (40%+)

a. **Set up the scene** exactly as described in the previous section.
b. Implement **keyboard control** for the sphere using **WASD** keys:
  i. 'W' moves the sphere forward;
  ii. 'S' moves the sphere backwards;
  iii. 'D' rotates the sphere clockwise;
  iv. 'A' rotates the sphere anti-clockwise;
  v. The game **should enter GameOver state** if the player moves into the water, and then stop.
c. Use kSphereSpeed and kRotationSpeed as **constants for storing** the speed at which you move and rotate the sphere. Choose a reasonable value that works well in the lab and on your machine.
d. Implement movement for your **top-down camera** by adding keyboard keys:
  i. 'Up' moves the camera forward along the world z-axis
  ii. 'Down' moves the camera backwards along the world z-axis
  iii. 'Right' moves the camera right along the world x-axis
  iv. 'Left' moves the camera left along the world x-axis
e. The P key causes the game to enter a **Paused state**. In the Paused state:
  i. Nothing should move
  ii. Hitting the P key causes the game to transition to the Playing state
f. In all game states, the player should be able to **press the Escape key to quit the game**.
g. The sphere should **pick up cubes** that are very close to it:
  i. Calculate the distance between the sphere and all cubes.
  ii. If the distance is not greater than the sum of the sphere's radius and half the width of the cube, the cube should be picked up by the sphere. For example, if the sphere's radius is 10 units, then any cube within (10 + 5/2) 12.5 units of the sphere should be picked up.
  iii. Picked-up cubes should not be visible (think of a way to make the cube disappear).
h. The code you submit MUST compile without errors.

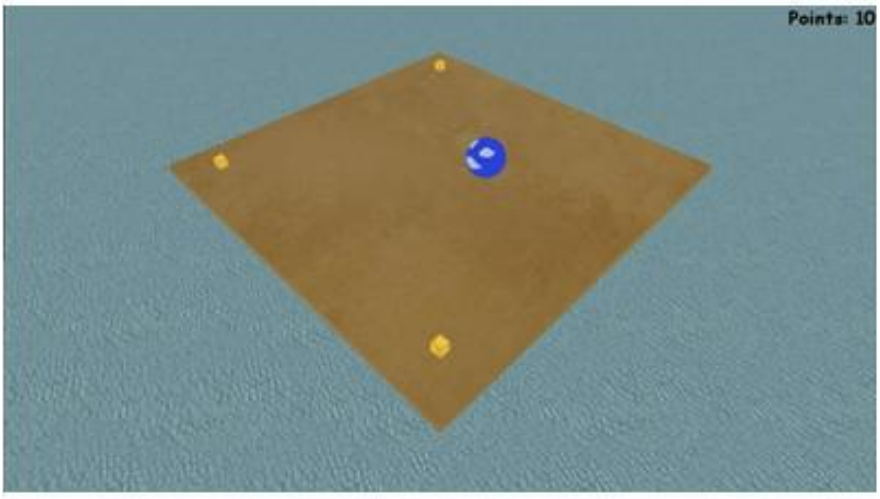
MILESTONE 2: LOWER SECOND CLASS (50%+)

a. Set up an **isometric camera** that can be activated by pressing the **"2" key**.
   - **i.** The camera should be positioned at (150, 150, -150), rotated by -45 degrees about its local y-axis **and then** by 45 degrees about its local x-axis (the order is important, why?).
   - **ii.** If the isometric camera is active, pressing the "1" key should reactivate the top-down camera.

b. **Reward the player with points** whenever the sphere picks up a cube
   - **i.** Create an integer variable named "playerPoints" for tracking points accumulated
   - **ii.** When the sphere "picks" up a cube, add 10 to the player's points.
   - **iii.** Display the player's points at the top and aligned to the right of the game window in the format "Points: X"; e.g., after picking up all four cubes, "Points: 40" should be displayed

c. When 4 of the cubes are picked up, **the sphere should grow in size**.
   - **i.** Implement this by scaling the sphere to 1.2 times its initial size; this will make the radius of the sphere 12 units.
   - **ii.** After scaling up, the y position of the sphere must be changed so that part of it does not appear below the floor (hint: consider the sphere's new radius).

d. Using the report template provided in Blackboard, tick the tasks that you manage to implement up to this point, and create a **State-Transition diagram** that explains the behavior of the game.



MILESTONE 3: UPPER SECOND CLASS (60%+)

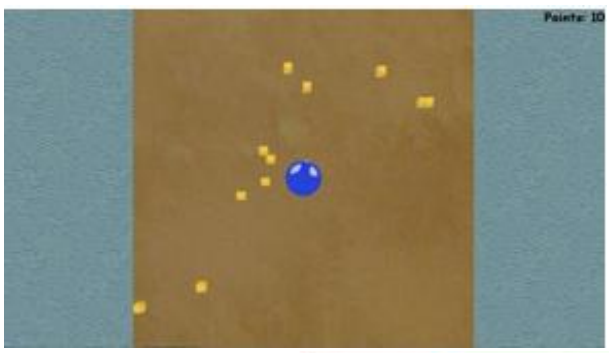
a. Rather than loading 4 cube models manually, **use an array of IModels** to create and store 12 cube models.
  i. Using a loop and a random number generating function, create and place all 12 cubes at random positions on the floor.
  ii. Consider using C++'s rand() function.
  iii. Does the scene look the same every time the game is executed? That is because rand() is a pseudo-random generator.

b. Update your cube pickup code to use **sphere-to-box collision detection** to check for contact between the sphere and the cubes. **The sphere-to-box should be a function**. Remember all the cubes are axis-aligned. Also remember that for every 4 cubes picked up, the sphere should be scaled to 1.2 times its size. **Note:** checking for collisions between the sphere and 12 cubes will lead to a reduced FPS rate; therefore, it is recommended that you utilize **variable timing** when moving your sphere from this point on.

c. When a player accumulates 120 points, the game should enter a **GameWon** state. In the GameWon state:
  i. The sphere should not be able to move.
  ii. A congratulatory message should be displayed at the center of the game window.

d. The game should now have 4 states: **Playing**, **Paused**, **GameWon,** and **GameOver**. You may also want to display an appropriate message in the GameOver state, while keeping the player fixed, instead of just stopping the game.

e. Update your report so it matches the current state of the game.







MILESTONE 4: FIRST CLASS (70%)

a. When randomly spawning cubes, check to ensure that **each cube is not within 10 units of other cubes or in collision with the sphere**. As long as (hint: while loop) the cube being spawned is within 10 units of another cube or in collision with the sphere, a new random position should be generated for the cube.

b. Instead of hiding picked-up cubes, **respawn them at a new random position** and ensure that the checks in 'a' above are performed. Consider using a **single function to spawn and respawn cubes**.

c. Implement a "**Hyper Mode**" in which the sphere attracts cubes within a 50-unit radius:

- **i.** Add a special cube using "**minicube.x**" and "**hypercube.jpg**" as its skin at a random position as in 'a' above.
- **ii.** On picking up the special cube, the sphere should transition into a "**Hyper**" state, and its skin should change to "**hypersphere.jpg**". The Hyper state should last for **5 seconds**, after which the sphere should return to its "**Regular**" state.
- **iii.** In the Hyper state, the sphere should **attract cubes that are within 50 units** of its center "**magnetically**"; attracted cubes should "**fly**" into the sphere before they are respawned.

d. Update your report so it matches the current state of the game.

MILESTONE 5: HIGHER FIRST (up to 100%)

a. Introduce an NPC to the game:
  i. Create an **enemy sphere** model and set its skin to **enemysphere.jpg**.
  ii. Choose a **random position far away from the player** sphere to position the enemy sphere.
  iii. Implement **automatic movement for the enemy sphere**, and set it up so that it can pick up cubes in the same manner as the player sphere. At a minimum, implement this so that the sphere appears to make random movement decisions e.g., whether to move up or down, left or right. Full marks will be awarded for implementing "intelligent" behavior.
  iv. Declare an integer variable called **enemyPoints** to keep track of the enemy sphere's points.
  v. If the enemy sphere collides with the player sphere, **they should bounce off each other** if the difference between their points is not greater than 40, otherwise, the sphere with the higher points **should pick up the other sphere**, earn 40 points, and **increase in size by a factor of 1.2**.

b. If the enemy sphere accumulates 120 points before the player sphere, or If the player sphere is picked up by the enemy sphere, then the game should enter a GameOver state.
  i. In the GameOver state, a game over message should be displayed on the centre of the screen, however, the enemy sphere should continue to move around as usual on the floor.

c. List the points of the player and NPC, highest point first, and aligned to the right of the screen.

d. Update your report so it matches the current state of the game.

If you manage to make it to this point, don't stop. Extra points will be given for additional features that will be implemented. If you decide to introduce new features, make sure that you mention these features in the report..!

MARKING SCHEME

Marks Breakdown			
No	**Feature**	**Points**	**Total**
R	Report	/5	**/5**
	Milestone 1		
1a	Basic scene setup	/20	**/49**
1b	Player Movement (WASD Keys) and GameOver state (step on water)	/6	
1c	kSphereSpeed and kRotationSpeed	/2	
1d	Camera movement on Top-down View (Up, Down, Left, Right)	/4	
1e	Pause and Playing states	/5	
1f	Key Escape terminates the game	/2	
1g	Cubes pick-up on collision	/10	
	Milestone 2		
2a	Isometric camera with Key_2 and Top-Down with Key_1	/2	**/7**
2b	Points (playerPoints variable; increase points by 10; display points)	/3	
2c	4 cubes picked, then increase the size of the sphere by 1.2	/2	
	Milestone 3		
3a	Array with 12 cubes that spawn at random locations	/3	**/10**
3b	Sphere-to-box collision function	/4	
3c	GameWon state when player picks 12 cubes (120 points)	/2	
3d	The game should have 4 states: Playing, Pause, GameWon, GameOver	/1	
	Milestone 4		
4a	Random spawn with a difference of 10 units	/3	**/15**
4b	Respawn when a cube is picked up in a different position	/4	
4c	Hyper-mode (add cube; change skin; pull cubes for 5 seconds)	/8	
	Milestone 5		
5ai	NPC sphere using the appropriate skin	/1	**/14**
5aii	Random position away from the player	/2	
5aiii	Automatic movement	/3	
5aiv	Enemy points visible	/1	
5av	Bounce on collision, BUT if the difference is >40, then the enemy should chase the player, scale every 40	/2	
5b	If the enemy has 120 points or the enemy eats the player, then the game enters GameOver state; the enemy should keep moving	/3	
5c	List the points based on the highest score	/2	
	Extra Points:		
	Total:	**/100**	